

开源软件基础

Python 基础

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

November 21, 2017



Outline

1 面向对象

2 函数式特性

3 函数式特性

- List Comprehension
- Map-Reduce

面向对象

面向对象三要素

- 继承：儿子点出父亲成员
- 封装：点号能点出成员
- 多态：儿子点出和父亲不同的行为

类的声明

```
class ClassName:  
    <statement-1>  
    .  
    .  
    .  
    <statement-N>
```

`__init__` 函数

```
def __init__(self):  
    self.data = []
```

Class Objects

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
```

成员变量和实例变量

```
class Dog:

    tricks = []           # mistaken use of a class

    def __init__(self, name):
        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d = Dog('Fido')
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
```

Class Objects

```
class MyClass:  
    """A simple example class"""  
    i = 12345  
  
    def f(self):  
        return 'hello world'
```


实例/类的成员

- 在 `__init__` 中用 `self`. 初始化的是实例成员
- 在类似 C++ 成员声明处初始化的是类成员（类似静态变量）
- 实例成员可以动态添加和删除
- 必要时用 `hasattr` 判断是否包含某成员

成员可见性

成员可见性

- 不存在类似 C++ 的私有
- 约定：下划线前缀的是实现细节
- ipython/Jupyter 默认不会补全
- 类似 *nix 下的 dotfiles

继承

```
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    pass

b = B()
b.foo()
```

多态

```
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    def foo(self):
        print("in B::foo")

b = B()
b.foo()
isinstance(b, B)
isinstance(b, A)
```

鸭子类型

在程序设计中，鸭子类型（英语：duck typing）是动态类型的一种风格。在这种风格中，一个对象有效的语义，不是由继承自特定的类或实现特定的接口，而是由“当前方法和属性的集合”决定。这个概念的名字来源于由 James Whitcomb Riley 提出的鸭子测试，“鸭子测试”可以这样表述：

鸭子测试

- If it walks like a duck
- It quacks like a duck
- Then it must be a duck

“当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子。”¹

¹https://en.wikipedia.org/wiki/Duck_typing

多态

```
class A:
    def foo(self):
        print("in A::foo")

class B(A):
    def foo(self):
        print("in B::foo")

class C:
    def foo(self):
        print("not subclass of A/B")

def test_class(a):
    a.foo()
```

Outline

1 面向对象

2 函数式特性

3 函数式特性

- List Comprehension
- Map-Reduce

函数一等公民

- 可以被存入变数或其他结构
- 可以被作为参数传递给其他函数
- 可以被作为函数的返回值
- 可以在执行期创造，而无需完全在设计期全部写出
- 即使没有被系结至某一名称，也可以存在

Lisp



Haskell

- 偏学术
- 号称最纯粹的函数式语言
- Avoid Haskell at all cost



Outline

- ① 面向对象
- ② 函数式特性
- ③ 函数式特性
 - List Comprehension
 - Map-Reduce

什么是函数式

函数式编程 (functional programming) 或称函数程序设计，是一种编程典范，它将电脑运算视为数学上的函数计算，并且避免使用程序状态以及易变对象。函数编程语言最重要的基础是 λ 演算 (lambda calculus)。而且 λ 演算的函数可以接受函数当作输入 (引数) 和输出 (传出值)。比起命令式编程，函数式编程更加强调程序执行的结果而非执行的过程，倡导利用若干简单的执行单元让计算结果不断渐进，逐层推导复杂的运算，而不是设计一个复杂的执行过程。

函数式特性

- First-class and higher-order functions
- Pure function
- Recursion
- Type system
- ...
- Referential transparency

λ 演算

λ 演算（英语：lambda calculus, λ -calculus）是一套从数学逻辑中发展，以变量绑定和替换的规则，来研究函数如何抽象化定义、函数如何被应用以及递归的形式系统。它由数学家阿隆佐·邱奇在 20 世纪 30 年代首次发表。Lambda 演算作为一种广泛用途的计算模型，可以清晰地定义什么是一个可计算函数，而任何可计算函数都能以这种形式表达和求值，它能模拟单一磁带图灵机的计算过程；尽管如此，Lambda 演算强调的是变换规则的运用，而非实现它们的具体机器。

λ 表达式

```
inc = lambda x: x + 1
```

```
add = lambda x, y: x + y
```

```
inc(5)
```

```
add(10, 20)
```

注意：Python的lambda表达式不能换行

Outline

- ① 面向对象
- ② 函数式特性
- ③ 函数式特性
 - List Comprehension
 - Map-Reduce

列表推导

```
[i * 2 for i in range(10)]
```

```
[i * 2 for i in range(10) if i % 2 == 0]
```

```
colours = ["red", "green", "yellow", "blue"]
```

```
things = ["house", "car", "tree"]
```

```
combined = [(x,y) for x in colours for y in things]
```

Outline

- ① 面向对象
- ② 函数式特性
- ③ 函数式特性
 - List Comprehension
 - Map-Reduce

Map/Filter/Reduce

```
map(lambda x: x ** 2, range(10))  
list(map(lambda x: x ** 2, range(10)))
```

```
filter(lambda x: x % 2 == 0,  
        map(lambda x: x ** 2, range(10)))
```

```
reduce(lambda x, y: x + y,  
        filter(lambda x: x % 2 == 0,  
                map(lambda x: x ** 2, range(10))))
```